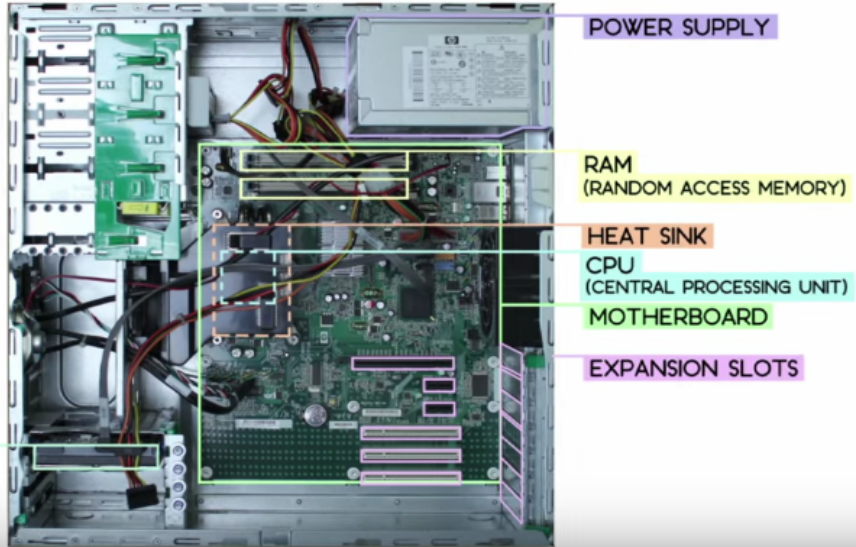


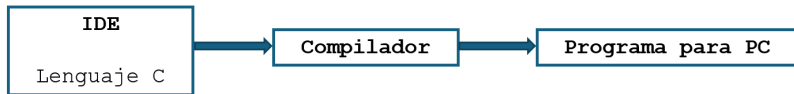
Tema 2: Fundamentos de C

Oscar Perpiñán Lamigueiro - David Álvarez

¿Qué hay dentro de un ordenador? (Inglés)



Hello World!



```
#include <stdio.h>

int main()
{
    printf("Hello World!");

    return 0;
}
```

Partes de un programa en C

```
/* Primero se incluyen las
   bibliotecas que se usan en el programa */
#include <stdio.h>

// Todo programa necesita una función main.
// Su contenido está delimitado entre llaves
int main()
{
    // La función printf muestra en pantalla el mensaje escrito entre comillas
    printf("Hello World!\n");

    // La función main devuelve un entero con return.
    return 0;

} // Aquí acaba main y por tanto el programa
```

Palabras clave en C

Modificadores de tipo	Tipos primitivos	Tipos datos avanzados	Sentencias de control
auto	char	enum	break
extern	double	struct	case
register	float	typedef	continue
static	int	union	do
const	long		else
volatile	short		if
signed	void		for
unsigned	sizeof		while
			return
			goto
			default
			switch

- 1 Datos en C
 - Almacenamiento de la información
 - Datos en un programa en C

- 2 Interacción entrada/salida (I/O)
 - Imprimir en el terminal
 - Lectura de datos desde el teclado

- 3 Operadores
 - Aritméticos
 - Comparadores
 - Lógicos

- 4 Otro tipo de datos, los caracteres
 - Caracteres

- 5 Conversión de tipos de datos

- 1 Datos en C
 - Almacenamiento de la información
 - Datos en un programa en C
- 2 Interacción entrada/salida (I/O)
 - Imprimir en el terminal
 - Lectura de datos desde el teclado
- 3 Operadores
 - Aritméticos
 - Comparadores
 - Lógicos
- 4 Otro tipo de datos, los caracteres
 - Caracteres
- 5 Conversión de tipos de datos

- Los ordenadores utilizan el sistema de numeración binario (dos dígitos, 0 y 1) para almacenar información.
- Un **dígito binario** (0 ó 1) se denomina *bit* (*binary digit*).
- Con **N bits** pueden representarse **2^N símbolos o 2^N números**
 - ▶ Ejemplo: con $N = 8$ bits se pueden representar los números positivos del 0 al 255 ($2^8 - 1$).

Representación de la información: binario y decimal

Ejemplo en decimal: 3452

10^3	10^2	10^1	10^0
1000	100	10	1
3	4	5	2

$$3452 = 3 \cdot 1000 + 4 \cdot 100 + 5 \cdot 10 + 2 \cdot 1$$

Ejemplo en binario: 10001111

2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
128	64	32	16	8	4	2	1
1	0	0	0	1	1	1	1

$$128 + 8 + 4 + 2 + 1 = 143$$

Unidades de almacenamiento

- Byte: 8 bits ($2^8 = 256$)
- Kilobyte (KB): 1024 bytes ($2^{10} = 1024$)
- Megabyte (MB): 1024 KB (2^{20} bytes = 2^{10} KB)
- Gigabyte (GB): 1024 MB (2^{30} bytes = 2^{10} MB = 2^{20} KB)
- ...

- 1 Datos en C
 - Almacenamiento de la información
 - Datos en un programa en C
- 2 Interacción entrada/salida (I/O)
 - Imprimir en el terminal
 - Lectura de datos desde el teclado
- 3 Operadores
 - Aritméticos
 - Comparadores
 - Lógicos
- 4 Otro tipo de datos, los caracteres
 - Caracteres
- 5 Conversión de tipos de datos

Tipos de datos

Tipo	Tamaño (bytes)	Rango (min)	Rango (max)
char	1	0	255
int	4	-2147483648	2147483647
float	4	$-3,4 * 10^{38}$	$3,4 * 10^{38}$

- Otros tipos

Constantes y Variables: declaración

- Son espacios de memoria que contienen datos. Se declaran para reservar la memoria en el ordenador.
- Declarar significa definir el tipo, asignarle un nombre y darle, si es necesario, un valor inicial.

Constantes datos cuyo valor *NO SE PUEDE MODIFICAR* (**const**).

const tipo nombre=valor_inicial;

Variables datos cuyo valor se puede modificar mediante el operador *asignación* (=)

tipo nombre=valor_inicial;

Nombres de constantes y variables

- Primer carácter: letra o carácter de subrayado (_) (**nunca un número**).
- Una o más letras (A-Z, a-z, *ñ excluida*), dígitos (0-9) o caracteres de subrayado.
- Tienen que ser distintos de las palabras clave.
- Las mayúsculas y las minúsculas son diferentes para el compilador.
- Es aconsejable que los nombres sean representativos

Definición de constantes y variables enteras

```
#include <stdio.h>
int main()
{
    // declara una variable ENTERA con el identificador total
    int total;
    // declara una constante ENTERA con el identificador num_alumnos
    int num_alumnos = 90;
    // declara una variable ENTERA num_prof, y le asigna el valor 2
    const int num_prof = 1;

    // asigna el valor de la constante c1 a la variable v1 (cambia su valor previo)
    total = num_prof;
    // asigna el valor de la constante c1 a la variable v2
    total = num_prof + num_alumnos;

    num_alumnos = 3;

    return 0;
}
```

Definición de constantes y variables reales

```
#include <stdio.h>
int main()
{
    // declara una variable REAL con el identificador area
    float area;
    // declara una constante REAL con el identificador c1
    const float pi = 3.141592;
    // declara una variable REAL v2, y le asigna el valor 2
    float radio = 5;

    // asigna el valor de la constante c1 a la variable v1 (cambia su valor previo)
    area = pi*radio*radio;

    radio = pi;
    area = pi*radio*radio;

    return 0;
}
```


- 1 Datos en C
 - Almacenamiento de la información
 - Datos en un programa en C
- 2 Interacción entrada/salida (I/O)
 - Imprimir en el terminal
 - Lectura de datos desde el teclado
- 3 Operadores
 - Aritméticos
 - Comparadores
 - Lógicos
- 4 Otro tipo de datos, los caracteres
 - Caracteres
- 5 Conversión de tipos de datos

- 1 Datos en C
 - Almacenamiento de la información
 - Datos en un programa en C
- 2 Interacción entrada/salida (I/O)
 - **Imprimir en el terminal**
 - Lectura de datos desde el teclado
- 3 Operadores
 - Aritméticos
 - Comparadores
 - Lógicos
- 4 Otro tipo de datos, los caracteres
 - Caracteres
- 5 Conversión de tipos de datos

Imprimir frases en el terminal

```
#include <stdio.h>

int main()
{
    // printf para imprimir una frase
    printf("El área de un círculo de radio 1, vale PI");

    // Y si lo queremos hacer con variables?

    return 0;
}
```

Especificadores de formato y secuencias de escape

Formato	Significado
%c	Carácter simple
%d	Entero decimal
%i	Entero decimal, octal o hexadecimal
%e	Coma flotante con exponente
%f	Coma flotante sin exponente
%g	Usa %e o %f, el más corto
%lf	Usar con tipo double
%o	Entero octal, sin el cero inicial
%s	Cadena de caracteres
%u	Entero decimal sin signo
%x	Entero hexadecimal sin 0x.

Secuencia	Significado
\n	Salto de línea
\t	Tabulador horizontal
\"	Comilla doble
\\	Barra invertida

Imprimir datos en el terminal

```
#include <stdio.h>

int main()
{
    float area;
    const float pi = 3.141592;
    int radio = 5;

    area = pi*radio*radio;

    // printf para imprimir una frase
    printf("El área de un círculo de radio %i, vale %f", radio, area);

    return 0;
}
```

- 1 Datos en C
 - Almacenamiento de la información
 - Datos en un programa en C
- 2 Interacción entrada/salida (I/O)
 - Imprimir en el terminal
 - Lectura de datos desde el teclado
- 3 Operadores
 - Aritméticos
 - Comparadores
 - Lógicos
- 4 Otro tipo de datos, los caracteres
 - Caracteres
- 5 Conversión de tipos de datos

Lectura de datos con scanf

scanf("formato", &variable);

```
#include <stdio.h>
int main()
{
    // Primero debo reservar la memoria donde guardar la información
    int num;

    // Pedir el dato por escrito es OPCIONAL, pero deseable
    printf("Escribe un número \t");

    //Atención: SCANF, la variable debe ir precedida de &
    scanf("%i", &num);

    printf("Has escrito el número %i\n", num);

    return 0;
}
```

Lectura distintos tipos de datos con scanf

Ejercicio

```
#include <stdio.h>
int main()
{
    int num_ent;
    float num_dec;

    printf("Escribe un numero \t");
    scanf("%i", &num_ent);
    ...
    // Rellenar el resto
    ...
    return 0;
}
```


Errores comunes con scanf

- Escribir dentro de la cadena de control mensajes y secuencias de escape (p.ej. `\n`).
- Olvidar poner el operador `&` delante de los argumentos cuando son variables de los tipos básicos (`int`, `float`, `char`)
- Poner un especificador de formato no compatible con el tipo del argumento.

- 1 Datos en C
 - Almacenamiento de la información
 - Datos en un programa en C
- 2 Interacción entrada/salida (I/O)
 - Imprimir en el terminal
 - Lectura de datos desde el teclado
- 3 Operadores
 - Aritméticos
 - Comparadores
 - Lógicos
- 4 Otro tipo de datos, los caracteres
 - Caracteres
- 5 Conversión de tipos de datos

- 1 Datos en C
 - Almacenamiento de la información
 - Datos en un programa en C
- 2 Interacción entrada/salida (I/O)
 - Imprimir en el terminal
 - Lectura de datos desde el teclado
- 3 Operadores
 - Aritméticos
 - Comparadores
 - Lógicos
- 4 Otro tipo de datos, los caracteres
 - Caracteres
- 5 Conversión de tipos de datos

Aritméticos

```
x + y  
x - y  
x / y // OJO con el tipo de datos  
x * y  
x % y // módulo o resto de división de entera
```

Aritméticos con enteros

```
#include <stdio.h>

int main()
{
    int x, y, sum;
    x = 10;
    y = 15;
    sum = x + y;

    printf("La suma de %i con %i es %i\n", x, y, sum);
    printf("La resta de %i con %i es %i\n", x, y, x-y);
    // los demás?
    // y con decimales?

    return 0;
}
```

Ejercicios

- Escriba un programa que lea del teclado una letra minúscula e imprima esa misma letra en mayúscula.
- Escriba un programa que pida un peso y altura de una persona y calcule su índice de masa corporal (imc). El imc se calcula dividiendo el peso en (kg) por la altura (m) al cuadrado.

Asignaciones especiales

- Recomendando *NO USAR* mientras se aprende

```
x += y // x = x + y
x -= y // x = x - y
x *= y // x = x * y
x /= y // x = x / y
x %= y // x = x % y
```

```
// ERROR: en el lado izquierdo no puede ir una expresión
x + y = 1
```

Ejercicio operaciones de asignación

```
#include <stdio.h>
int main() {
    int a, b = 3;

    a = 5;
    printf("a = %i\n", a);
    a *= 4; // a = a * 4
    printf("a = %i\n", a);
    a += b; // a = a + b
    printf("a = %i\n", a);
    a /= (b + 1); // a = a / (b+1)
    printf("a = %i\n", a);
    a = b = 1;
    printf("a = %i, b = %i\n", a, b);

    return 0;
}
```


Incrementales - solo usar en bucles

```
y = ++x // x = x + 1; y = x (preincremento)
y = x++ // y = x; x = x + 1 (postincremento)

y = --x // x = x - 1; y = x (predecremento)
y = x-- // y = x; x = x - 1 (postdecremento)
```

Ejercicio operaciones de incremento

```
#include <stdio.h>

int main()
{
    int b = 2, r;
    //Preincremento
    r = ++b;
    printf("b = %i, r = %i\n", b, r);
    //Postincremento
    r = b++;
    printf("b = %i, r = %i\n", b, r);

    return 0;
}
```

Ejercicio operaciones de incremento 2

```
#include <stdio.h>

int main()
{
    int a = 0;
    printf("a = %i\n", ++a);
    printf("a = %i\n", a++);
    printf("a = %i\n", a);
    printf("a = %i\n", --a);
    printf("a = %i\n", a--);
    printf("a = %i\n", a);

    return 0;
}
```

Ejercicio precedencia y asociatividad

```
#include <stdio.h>

int main() { // Y si son enteros?
    float a = 4, b = 7, c = 3, g = 9, result;

    result = a + b * c;
    printf( "resultado = %f\n", result);

    result = (a + b) * c;
    printf( "resultado = %f\n", result);

    result = a * b / c * g;
    printf("resultado = %f\n", result);

    result = (a * b) / (c * g);
    printf("resultado = %f\n", result);

    return 0;
}
```

- 1 Datos en C
 - Almacenamiento de la información
 - Datos en un programa en C
- 2 Interacción entrada/salida (I/O)
 - Imprimir en el terminal
 - Lectura de datos desde el teclado
- 3 Operadores
 - Aritméticos
 - **Comparadores**
 - Lógicos
- 4 Otro tipo de datos, los caracteres
 - Caracteres
- 5 Conversión de tipos de datos

Relacionales

- El resultado es verdadero o falso.
- En programación esto significa 1 ó 0 respectivamente.

```
x == y  
x != y  
x > y  
x >= y  
x < y  
x <= y
```

Ejercicio operaciones comparadores

```
#include <stdio.h>
int main()
{
    int x = 10, y = 3;

    printf("x igual a y = %i\n", (x == y));
    printf("x distinto a y = %i\n", (x != y));
    printf("x mayor que y = %i\n", (x > y));
    printf("x menor o igual a y = %i\n", (x <= y));
    printf("x mayor o igual que y = %i\n", (x >= y));

    return 0;
}
```

- 1 Datos en C
 - Almacenamiento de la información
 - Datos en un programa en C
- 2 Interacción entrada/salida (I/O)
 - Imprimir en el terminal
 - Lectura de datos desde el teclado
- 3 Operadores
 - Aritméticos
 - Comparadores
 - Lógicos
- 4 Otro tipo de datos, los caracteres
 - Caracteres
- 5 Conversión de tipos de datos

- AND, OR, NOT

```
x && y //AND  
x || y //OR  
!x //NOT, operador unario
```

- Operador condicional ? (ternario)

```
// expresión booleana ? valor si cierto : valor si falso  
x > y ? 1 : 0  
x == y ? 'v' : 'f'
```

Ejercicio operaciones lógicas

```
#include <stdio.h>
int main()
{
    int x, resto;

    printf("Escribe un número entero: ");

    scanf("%i", &x);

    // Calcula el resto de dividir por 2
    resto = x % 2;

    //Imprime una P si es par 0, una I si es impar
    printf("Es un número %c\n", (resto == 0) ? 'P' : 'I');

    return 0;
}
```

- 1 Datos en C
 - Almacenamiento de la información
 - Datos en un programa en C
- 2 Interacción entrada/salida (I/O)
 - Imprimir en el terminal
 - Lectura de datos desde el teclado
- 3 Operadores
 - Aritméticos
 - Comparadores
 - Lógicos
- 4 Otro tipo de datos, los caracteres
 - Caracteres
- 5 Conversión de tipos de datos

- 1 Datos en C
 - Almacenamiento de la información
 - Datos en un programa en C
- 2 Interacción entrada/salida (I/O)
 - Imprimir en el terminal
 - Lectura de datos desde el teclado
- 3 Operadores
 - Aritméticos
 - Comparadores
 - Lógicos
- 4 Otro tipo de datos, los caracteres
 - Caracteres
- 5 Conversión de tipos de datos

No sólo números - caracteres

- Cualquier información puede representarse con un conjunto de bits.
- ASCII (American Standard Code for Information Interchange): Estándar de 7 bits (128 caracteres), 95 caracteres imprimibles (del 32 al 126).

01001000	01101111	01101100	01100001
72	111	108	97
H	o	l	a

Tabla ASCII (no está entera)

32		44	,	56	8	68	D	80	P	92	\	104	h	116	t
33	!	45	-	57	9	69	E	81	Q	93	]	105	i	117	u
34	"	46	.	58	:	70	F	82	R	94	^	106	j	118	v
35	#	47	/	59	;	71	G	83	S	95	_	107	k	119	w
36	\$	48	0	60	<	72	H	84	T	96	`	108	l	120	x
37	%	49	1	61	=	73	I	85	U	97	a	109	m	121	y
38	&	50	2	62	>	74	J	86	V	98	b	110	n	122	z
39	'	51	3	63	?	75	K	87	W	99	c	111	o	123	{
40	(	52	4	64	@	76	L	88	X	100	d	112	p	124	
41	)	53	5	65	A	77	M	89	Y	101	e	113	q	125	}
42	*	54	6	66	B	78	N	90	Z	102	f	114	r	126	~
43	+	55	7	67	C	79	O	91	[	103	g	115	s		

Definición de constantes y variables de tipo caracter

```
#include <stdio.h>

int main()
{
    //declara una variable de tipo CHAR con el identificador letra
    char letra = 'z';
    // declara una constante CHAR con el identificador c1
    const char c1 = '8';
    // asigna el valor de la constante c1 a la variable letra
    letra = c1;
    // Y char con un número? SI. Usando la Tabla ASCII. Qué caracter es?
    letra = 122;
    // es esto válido?
    c1 = 122;

    return 0;
}
```

Especificadores de formato y secuencias de escape

Formato	Significado
%c	Carácter simple
%d	Entero decimal
%i	Entero decimal, octal o hexadecimal
%e	Coma flotante con exponente
%f	Coma flotante sin exponente
%g	Usa %e o %f, el más corto
%lf	Usar con tipo double
%o	Entero octal, sin el cero inicial
%s	Cadena de caracteres
%u	Entero decimal sin signo
%x	Entero hexadecimal sin 0x.

Secuencia	Significado
\n	Salto de línea
\t	Tabulador horizontal
\"	Comilla doble
\\	Barra invertida

Imprimir datos en el terminal

```
#include <stdio.h>

int main()
{
    // símbolo
    char sim = 'H';
    // número atómico
    int num_at = 1;
    // masa atómica
    float masa_at = 1.00794;

    // printf para imprimir una frase
    printf("El símbolo del hidrógeno es %c, su número atómico es %i, y su masa atómica es %f", sim, num_at, masa_at);

    return 0;
}
```

Imprimir datos en el terminal, números y caracteres

```
int main()
{

    char sim = 'H'; // símbolo
    int num_at = 1; // número atómico
    float masa_at = 1.00794; // masa atómica

    // printf para imprimir una frase
    printf("El símbolo del hidrógeno es %c, su número atómico es %i, y su masa atómica es %f\n", sim, num_at, masa_at);

    // OJO!! Controlamos cuantos decimales aparecen
    printf("El símbolo del yodo es %c, su número atómico es %i, y su masa atómica es %.3f", sim+1, num_at+52, 126.90447);

    return 0;
}
```

Lectura de caracteres, scanf y getchar

```
scanf("%c", &variable);
```

```
variable = getchar();
```

- **scanf** vale para cualquier tipo de datos, mientras que **getchar** solo sirve para leer caracteres.
- ¿Qué pasa cuando queremos leer 2 caracteres de manera consecutiva?

Ejercicio: leer los caracteres de tu nombre para luego imprimirlos por pantalla. También leerlos como **int** e imprimirlos como **char**, y al revés.

Lectura de varios caracteres seguidos, fflush

```
#include <stdio.h>
int main()
{
    // Primero debo reservar la memoria donde guardar la información
    char ejemplo1, ejemplo2;

    // Pedir el dato por escrito es OPCIONAL, pero deseable
    printf("Escribe un caracter \t");
    ejemplo1 = getchar();

    printf("Escribe otro caracter \t");
    fflush(stdin);
    scanf("%c", &ejemplo2);

    printf("Has los caracteres %c y %c\n", ejemplo1, ejemplo2);

    return 0;
}
```

- 1 Datos en C
 - Almacenamiento de la información
 - Datos en un programa en C
- 2 Interacción entrada/salida (I/O)
 - Imprimir en el terminal
 - Lectura de datos desde el teclado
- 3 Operadores
 - Aritméticos
 - Comparadores
 - Lógicos
- 4 Otro tipo de datos, los caracteres
 - Caracteres
- 5 Conversión de tipos de datos

Conversión implícita

Asignaciones el valor de la derecha se convierte al tipo de la variable de la izquierda (posible aviso o error). NO RECOMENDABLE.

```
#include <stdio.h>

int main()
{
    float f1 = 3.7, f2;
    int i1 = 2, i2;
    // Real a entero: pierde decimales
    i2 = f1;
    printf("Un real %f convertido a entero %i\n", f1, i2);
    // Entero a real: no cambia valor
    f2 = i1;
    printf("Un entero %i convertido a real %f\n", i1, f2);

    return 0;
}
```

Conversión explícita

Conversión explícita o forzada (tipo) expresión

```
#include <stdio.h>

int main()
{
    int i1 = 5, i2=2;
    float f1;

    printf("La división entre enteros: %i/%i = %f\n", i1, i2, (float)(i1/i2));

    // Completar el último campo
    //printf("El resto de %i entre %i es: %i", i1, f1, ...);

    return 0;
}
```

Conversión en expresiones

Expresiones los valores de los operandos se convierten al tipo del operando que tenga la precisión más alta.

```
#include <stdio.h>

int main()
{
    double f1 = 100;
    int i1 = 150, i2 = 100;
    printf("Un entero, %i, dividido por un real, %f,", i1, f1);
    printf(" produce un real, %f\n", i1 / f1);
    printf("Un entero, %i, por un entero, %i: %i\n", i1, i2, i1 / i2);

    return 0;
}
```